PROGRAMAÇÃO WEB

Tutorial Frontend

Autenticação JWT com HTML e JavaScript Puro

1.º Ano | Nível Básico | Complemento da Ficha de Autenticação

Docentes:

Célia Soares (P) Email: celiasoares@estg.ipvc.pt

Marcelo Antunes Fernandes (Docente Responsável - T) Email: fernandesmarcelo@estg.ipvc.pt

Introdução

Neste tutorial vais criar uma interface web simples para a API de autenticação que desenvolveste na ficha de trabalho. Sem frameworks, sem instalações — apenas três ficheiros HTML que abres diretamente no browser.

O objetivo é perceber na prática como o frontend e o backend comunicam: como se envia um formulário para uma API, onde se guarda o token JWT, e como se usa esse token para aceder a rotas protegidas.

> **ℹ Nota**
> Não precisas de instalar nada para este tutorial.
> Basta ter o servidor backend a correr e um browser. Abre cada ficheiro .html diretamente no browser ou usa a extensão Live Server do VS Code.

O que vais construir

Três ficheiros HTML independentes, cada um com uma responsabilidade clara:

Ficheiro	Responsabilidade
`registo.html`	Formulário para criar uma conta nova (POST /auth/signup)
`login.html`	Formulário de login que guarda o token (POST /auth/signin)
`tasks.html`	Lista de tarefas — usa o token para aceder à API protegida

Como funciona o JWT no frontend

Antes de escrever código, é importante perceber o fluxo:

1. O utilizador preenche o formulário de login e clica em Entrar
2. O browser envia os dados (email + password) para POST /auth/signin
3. O servidor valida as credenciais e devolve um token JWT
4. O browser guarda esse token no localStorage
5. Em cada pedido seguinte a rotas protegidas, o token é enviado no header Authorization
6. O servidor valida o token e responde com os dados pedidos

◉ Conceito

O que é o localStorage?
É um espaço de armazenamento no próprio browser, persistente entre sessões.
Guardas um valor com: localStorage.setItem('token', valor)
Lês com: localStorage.getItem('token')
Apagas com: localStorage.removeItem('token')
O token fica guardado mesmo que feches e reabras o browser.

PASSO 1

Estrutura de Ficheiros e Estilos Base

Criar a pasta do projeto e os ficheiros necessários

1.1 Criar a estrutura do projeto

Cria uma pasta chamada frontend-auth com os seguintes ficheiros:

```
frontend-auth/
├── registo.html
├── login.html
├── tasks.html
└── estilo.css
```

✓ Dica

Podes criar os ficheiros no VS Code. Não precisas de npm init nem de qualquer instalação — são ficheiros simples.

1.2 Folha de estilos partilhada — estilo.css

Este ficheiro de estilos vai ser partilhado pelas três páginas. Cria o ficheiro estilo.css com o seguinte conteúdo:

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f0f4f8;
  color: #222;
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 2rem;
}

.cartao {
  background: #fff;
  border-radius: 12px;
  padding: 2rem;
  width: 100%;
```

```
  max-width: 420px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.08);
}

.cartao-largo {
  max-width: 700px;
}

h1 {
  font-size: 1.6rem;
  color: #1F4E8C;
  margin-bottom: 0.3rem;
}

.subtitulo {
  color: #888;
  font-size: 0.95rem;
  margin-bottom: 1.8rem;
}

label {
  display: block;
  font-size: 0.9rem;
  font-weight: 600;
  color: #444;
  margin-bottom: 0.3rem;
}

input, textarea {
  width: 100%;
  padding: 0.65rem 0.9rem;
  border: 1.5px solid #ddd;
  border-radius: 8px;
  font-size: 1rem;
  font-family: inherit;
  margin-bottom: 1.1rem;
  transition: border-color 0.2s;
}

input:focus, textarea:focus {
  outline: none;
  border-color: #2E75B6;
}

button {
  width: 100%;
  padding: 0.75rem;
  background: #2E75B6;
  color: #fff;
  border: none;
```

```
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s;
}

button:hover    { background: #1F4E8C; }
button:disabled { background: #aaa; cursor: not-allowed; }

button.perigo {
  background: transparent;
  color: #c0392b;
  border: 1.5px solid #c0392b;
  padding: 0.3rem 0.8rem;
  width: auto;
  font-size: 0.85rem;
}
button.perigo:hover { background: #c0392b; color: #fff; }

.mensagem {
  padding: 0.75rem 1rem;
  border-radius: 8px;
  font-size: 0.9rem;
  margin-bottom: 1rem;
  display: none;
}
.mensagem.erro   { background: #fde8e8; color: #922b21; display: block; }
.mensagem.sucesso{ background: #e2f0e8; color: #1D6A3A; display: block; }

.ligacao {
  text-align: center;
  margin-top: 1.2rem;
  font-size: 0.9rem;
  color: #888;
}
.ligacao a { color: #2E75B6; text-decoration: none; font-weight: 600; }
.ligacao a:hover { text-decoration: underline; }

/* Tasks */
.cabecalho-tasks {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1.5rem;
}

.tag-utilizador {
  font-size: 0.85rem;
  background: #e8f0fe;
```

```
    color: #1F4E8C;
    padding: 0.3rem 0.8rem;
    border-radius: 20px;
  }

  button.sair {
    width: auto;
    background: transparent;
    color: #888;
    border: 1.5px solid #ddd;
    padding: 0.3rem 0.8rem;
    font-size: 0.85rem;
    font-weight: 400;
  }
  button.sair:hover { background: #f5f5f5; color: #333; }

  .formulario-nova-task {
    display: flex;
    gap: 0.5rem;
    margin-bottom: 1.5rem;
  }
  .formulario-nova-task input { margin-bottom: 0; }
  .formulario-nova-task button { width: auto; padding: 0.65rem 1.2rem; }

  .lista-tasks { list-style: none; }

  .task-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 0.85rem 1rem;
    border: 1.5px solid #eee;
    border-radius: 8px;
    margin-bottom: 0.6rem;
    background: #fafafa;
  }
  .task-item:hover { border-color: #d0e0f0; background: #f5f9ff; }

  .task-titulo { font-size: 1rem; color: #222; }
  .vazia { color: #bbb; text-align: center; padding: 2rem 0; font-size: 0.95rem;
  }
```

ℹ Nota

Este ficheiro de estilos vai ser referenciado pelas três páginas HTML. Mantém-no na mesma pasta.

PASSO 2

Página de Registo

registo.html — formulário para criar uma conta

2.1 Estrutura HTML

Cria o ficheiro registo.html. Começa pela estrutura base de qualquer página HTML:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Registar — API Tasks</title>
  <link rel="stylesheet" href="estilo.css" />
</head>
<body>

  <div class="cartao">
    <h1>Criar conta</h1>
    <p class="subtitulo">Preenche os campos para te registares.</p>

    <!-- Caixa de mensagem (erro ou sucesso) -->
    <div id="mensagem" class="mensagem"></div>

    <form id="form-registo">
      <label for="nome">Nome</label>
      <input type="text" id="nome" placeholder="O teu nome" required />

      <label for="email">Email</label>
      <input type="email" id="email" placeholder="email@exemplo.com" required
/>

      <label for="password">Password</label>
      <input type="password" id="password" placeholder="Mínimo 6 caracteres"
required />

      <button type="submit" id="btn-registar">Criar conta</button>
    </form>

    <div class="ligacao">
      Já tens conta? <a href="login.html">Entrar</a>
    </div>
  </div>

  <script src="registo.js"></script>
</body>
```

```
</html>
```

◉ Conceito

O que é o DOM?

O DOM (Document Object Model) é a representação do HTML em memória que o browser cria quando abre uma página.

Com JavaScript consegues aceder e modificar qualquer elemento pelo seu id:

document.getElementById('mensagem') → devolve o elemento com id="mensagem"
elemento.textContent = 'Texto' → altera o texto do elemento
elemento.className = 'mensagem erro'→ altera a classe CSS do elemento

2.2 JavaScript do registo — registo.js

Cria o ficheiro registo.js na mesma pasta. Este ficheiro trata do envio do formulário para a API:

```
// Selecionar elementos do DOM pelo seu id
const form      = document.getElementById('form-registo');
const msgDiv    = document.getElementById('mensagem');
const btnSubmit = document.getElementById('btn-registar');

// URL base do backend — ajusta se a porta for diferente
const API = 'http://localhost:3000';

// Função auxiliar: mostra uma mensagem de erro ou sucesso
function mostrarMensagem(texto, tipo) {
  msgDiv.textContent = texto;
  msgDiv.className   = 'mensagem ' + tipo; // 'erro' ou 'sucesso'
}

// Ouvir o evento 'submit' do formulário
form.addEventListener('submit', async function (evento) {
  // Impede o comportamento padrão do browser (recarregar a página)
  evento.preventDefault();

  // Ler os valores dos campos
  const nome     = document.getElementById('nome').value.trim();
  const email    = document.getElementById('email').value.trim();
  const password = document.getElementById('password').value;

  // Desativar o botão enquanto o pedido está a ser feito
  btnSubmit.disabled    = true;
  btnSubmit.textContent = 'A criar conta...';

  try {
    // Fazer o pedido POST à API
    const resposta = await fetch(API + '/auth/signup', {
      method:  'POST',
```

```
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ name: nome, email, password }),
    });

    // Converter a resposta para JSON
    const dados = await resposta.json();

    if (!resposta.ok) {
      // O servidor devolveu um erro (ex: email já existe)
      mostrarMensagem(dados.message || 'Erro ao criar conta.', 'erro');
      return;
    }

    // Sucesso — mostrar mensagem e redirecionar para o login
    mostrarMensagem('Conta criada! A redirecionar...', 'sucesso');
    setTimeout(() => { window.location.href = 'login.html'; }, 1500);

  } catch (erro) {
    // Erro de rede (ex: servidor não está a correr)
    mostrarMensagem('Não foi possível ligar ao servidor.', 'erro');
  } finally {
    // Reativar o botão independentemente do resultado
    btnSubmit.disabled    = false;
    btnSubmit.textContent = 'Criar conta';
  }
});
```

✓ Dica

Abre o browser, carrega registo.html e testa o formulário.
Abre também o separador Network nas ferramentas de desenvolvimento (F12) para veres o pedido HTTP a ser enviado e a resposta da API.

⚠ Atenção

Atenção ao nome dos campos no JSON!
O body que envias tem de corresponder exatamente ao que o backend espera.
Verifica no teu controller de signup quais os campos que são lidos do req.body.

PASSO 3

Página de Login

login.html — autenticar e guardar o token JWT

3.1 Estrutura HTML

Cria o ficheiro login.html:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Entrar — API Tasks</title>
  <link rel="stylesheet" href="estilo.css" />
</head>
<body>

  <div class="cartao">
    <h1>Entrar</h1>
    <p class="subtitulo">Usa as tuas credenciais para aceder.</p>

    <div id="mensagem" class="mensagem"></div>

    <form id="form-login">
      <label for="email">Email</label>
      <input type="email" id="email" placeholder="email@exemplo.com" required
/>

      <label for="password">Password</label>
      <input type="password" id="password" placeholder="A tua password"
required />

      <button type="submit" id="btn-entrar">Entrar</button>
    </form>

    <div class="ligacao">
      Ainda não tens conta? <a href="registo.html">Criar conta</a>
    </div>
  </div>

  <script src="login.js"></script>
</body>
</html>
```

3.2 JavaScript do login — login.js

Cria o ficheiro login.js. Esta é a parte mais importante do tutorial — é aqui que o token é recebido e guardado:

```
const form      = document.getElementById('form-login');
const msgDiv    = document.getElementById('mensagem');
const btnSubmit = document.getElementById('btn-entrar');

const API = 'http://localhost:3000';

function mostrarMensagem(texto, tipo) {
  msgDiv.textContent = texto;
  msgDiv.className   = 'mensagem ' + tipo;
}

// Se já existe token guardado, redirecionar diretamente para tasks
if (localStorage.getItem('token')) {
  window.location.href = 'tasks.html';
}

form.addEventListener('submit', async function (evento) {
  evento.preventDefault();

  const email    = document.getElementById('email').value.trim();
  const password = document.getElementById('password').value;

  btnSubmit.disabled    = true;
  btnSubmit.textContent = 'A entrar...';

  try {
    const resposta = await fetch(API + '/auth/signin', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ email, password }),
    });

    const dados = await resposta.json();

    if (!resposta.ok) {
      mostrarMensagem(dados.message || 'Credenciais inválidas.', 'erro');
      return;
    }

    // ─── PASSO FUNDAMENTAL ──────────────────────────────
    // O servidor devolveu o token — guardamo-lo no localStorage
    localStorage.setItem('token', dados.token);
    // ────────────────────────────────────────────────────
```

```
      mostrarMensagem('Login com sucesso! A redirecionar...', 'sucesso');
      setTimeout(() => { window.location.href = 'tasks.html'; }, 1000);

    } catch (erro) {
      mostrarMensagem('Não foi possível ligar ao servidor.', 'erro');
    } finally {
      btnSubmit.disabled    = false;
      btnSubmit.textContent = 'Entrar';
    }
  });
```

◉ Conceito

O que acontece quando fazes localStorage.setItem('token', dados.token)?
Estás a guardar o token JWT no browser. A partir deste momento, qualquer página do mesmo domínio pode ler esse token com localStorage.getItem('token').
Para veres o token guardado: F12 > Application > Local Storage > localhost.

✓ Dica

Testa o login com credenciais corretas e incorretas.
Após o login bem-sucedido, abre o F12 > Application > Local Storage e confirma que o token aparece guardado.

PASSO 4

Página de Tasks

tasks.html — consumir rotas protegidas com o token

4.1 Estrutura HTML

Cria o ficheiro tasks.html. Esta página é mais rica — tem cabeçalho, formulário de criação e lista de tarefas:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>As minhas tasks — API Tasks</title>
  <link rel="stylesheet" href="estilo.css" />
</head>
<body>

  <div class="cartao cartao-largo">

    <!-- Cabeçalho com nome do utilizador e botão de sair -->
    <div class="cabecalho-tasks">
      <h1>As minhas tasks</h1>
      <div style="display: flex; gap: 0.6rem; align-items: center;">
        <span id="nome-utilizador" class="tag-utilizador"></span>
        <button class="sair" id="btn-sair">Sair</button>
      </div>
    </div>

    <div id="mensagem" class="mensagem"></div>

    <!-- Formulário para criar nova task -->
    <div class="formulario-nova-task">
      <input type="text" id="nova-task" placeholder="Escreve uma nova
tarefa..." />
      <button id="btn-adicionar">Adicionar</button>
    </div>

    <!-- Lista de tasks (preenchida pelo JavaScript) -->
    <ul id="lista-tasks" class="lista-tasks">
      <li class="vazia">A carregar tarefas...</li>
    </ul>

  </div>

  <script src="tasks.js"></script>
```

```
</body>
</html>
```

4.2 JavaScript das tasks — tasks.js

Este ficheiro contém toda a lógica da página: verificar autenticação, carregar tasks, criar e apagar. Lê com atenção os comentários — cada bloco está explicado:

```
const API          = 'http://localhost:3000';
const listaEl      = document.getElementById('lista-tasks');
const msgDiv       = document.getElementById('mensagem');
const nomeEl       = document.getElementById('nome-utilizador');
const inputNova    = document.getElementById('nova-task');
const btnAdicionar = document.getElementById('btn-adicionar');
const btnSair      = document.getElementById('btn-sair');

// ── 1. Verificar se o utilizador está autenticado ──────────────
// Se não há token, redirecionar para o login imediatamente
const token = localStorage.getItem('token');
if (!token) {
  window.location.href = 'login.html';
}

// ── 2. Função auxiliar: pedidos autenticados ───────────────────
// Esta função adiciona sempre o header Authorization com o token
async function pedidoAPI(endpoint, opcoes = {}) {
  const resposta = await fetch(API + endpoint, {
    ...opcoes,
    headers: {
      'Content-Type':  'application/json',
      'Authorization': 'Bearer ' + token,  // <- o token é enviado aqui
      ...opcoes.headers,
    },
  });

  // Se o servidor devolver 401, o token expirou ou é inválido
  if (resposta.status === 401) {
    localStorage.removeItem('token');
    window.location.href = 'login.html';
    return;
  }

  return resposta;
}

// ── 3. Mostrar mensagem de feedback ────────────────────────────
function mostrarMensagem(texto, tipo) {
  msgDiv.textContent = texto;
  msgDiv.className   = 'mensagem ' + tipo;
```

```
  setTimeout(() => { msgDiv.className = 'mensagem'; }, 3000);
}

// ── 4. Carregar o perfil do utilizador ──────────────────────
async function carregarPerfil() {
  try {
    const resposta = await pedidoAPI('/auth/profile');
    if (!resposta) return;
    const dados = await resposta.json();
    nomeEl.textContent = dados.name || dados.email;
  } catch (e) {
    console.error('Erro ao carregar perfil:', e);
  }
}

// ── 5. Renderizar a lista de tasks no HTML ──────────────────
function renderizarTasks(tasks) {
  listaEl.innerHTML = ''; // Limpar a lista atual

  if (tasks.length === 0) {
    listaEl.innerHTML = '<li class="vazia">Ainda não tens tarefas. Adiciona uma
acima!</li>';
    return;
  }

  tasks.forEach(function (task) {
    // Criar o elemento <li> para cada task
    const li = document.createElement('li');
    li.className = 'task-item';

    li.innerHTML = `
      <span class="task-titulo">${task.title}</span>
      <button class="perigo" data-id="${task.id}">Apagar</button>
    `;

    // Adicionar evento de clique ao botão Apagar
    li.querySelector('button').addEventListener('click', function () {
      apagarTask(task.id);
    });

    listaEl.appendChild(li);
  });
}

// ── 6. Buscar todas as tasks à API ──────────────────────────
async function carregarTasks() {
  listaEl.innerHTML = '<li class="vazia">A carregar...</li>';
  try {
    const resposta = await pedidoAPI('/tasks');
    if (!resposta) return;
```

```
    const dados = await resposta.json();
    renderizarTasks(dados);
  } catch (e) {
    mostrarMensagem('Erro ao carregar tarefas.', 'erro');
  }
}

// ── 7. Criar uma nova task ──────────────────────────────
async function criarTask() {
  const titulo = inputNova.value.trim();
  if (!titulo) return; // Ignorar se o campo estiver vazio

  btnAdicionar.disabled    = true;
  btnAdicionar.textContent = '...';

  try {
    const resposta = await pedidoAPI('/tasks', {
      method: 'POST',
      body:   JSON.stringify({ title: titulo }),
    });
    if (!resposta) return;

    if (!resposta.ok) {
      const dados = await resposta.json();
      mostrarMensagem(dados.message || 'Erro ao criar tarefa.', 'erro');
      return;
    }

    inputNova.value = ''; // Limpar o campo de input
    await carregarTasks(); // Recarregar a lista
    mostrarMensagem('Tarefa criada!', 'sucesso');

  } catch (e) {
    mostrarMensagem('Erro ao criar tarefa.', 'erro');
  } finally {
    btnAdicionar.disabled    = false;
    btnAdicionar.textContent = 'Adicionar';
  }
}

// ── 8. Apagar uma task ──────────────────────────────────
async function apagarTask(id) {
  if (!confirm('Tens a certeza que queres apagar esta tarefa?')) return;

  try {
    const resposta = await pedidoAPI('/tasks/' + id, { method: 'DELETE' });
    if (!resposta) return;

    if (!resposta.ok) {
      mostrarMensagem('Erro ao apagar tarefa.', 'erro');
```

```
      return;
    }

    await carregarTasks(); // Recarregar a lista
    mostrarMensagem('Tarefa apagada.', 'sucesso');

  } catch (e) {
    mostrarMensagem('Erro ao apagar tarefa.', 'erro');
  }
}

// ── 9. Eventos de interação ──────────────────────────
// Botão Adicionar
btnAdicionar.addEventListener('click', criarTask);

// Premir Enter no campo de input também cria a task
inputNova.addEventListener('keydown', function (e) {
  if (e.key === 'Enter') criarTask();
});

// Botão Sair — apaga o token e vai para o login
btnSair.addEventListener('click', function () {
  localStorage.removeItem('token');
  window.location.href = 'login.html';
});

// ── 10. Inicializar a página ─────────────────────────
carregarPerfil();
carregarTasks();
```

◉ Conceito

Como é que o token chega ao servidor?
Em cada pedido a uma rota protegida, enviamos o header:
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
O middleware authenticateToken do backend lê este header,
extrai o token, valida-o com jwt.verify() e só então deixa o pedido passar.

PASSO 5

CORS e Teste Final

Ligar o frontend ao backend e verificar tudo

5.1 Configurar CORS no backend

O browser bloqueia pedidos entre origens diferentes por segurança. Como o frontend está numa porta diferente do backend, precisas de configurar o CORS no servidor Node.js:

```
# No projeto backend
npm install cors
```

Adiciona o middleware ao app.js ou server.js do backend, antes de qualquer rota:

```
const cors = require('cors');

// Permitir pedidos do frontend
app.use(cors({
  origin: '*',  // Em desenvolvimento aceita qualquer origem
}));

// Esta linha deve vir ANTES de app.use(express.json()) e das rotas
```

⚠ Atenção

origin: '*' é adequado apenas em desenvolvimento.
Em produção, substitui pelo URL real do teu frontend — por exemplo: origin: 'https://meusite.com'

5.2 Confirmar os nomes das rotas do backend

Este tutorial usa os seguintes endpoints. Confirma que o teu backend os tem definidos com exatamente estes paths:

Método	Rota	Protegida?	Utilização no frontend
POST	/auth/signup	Não	Criar conta (registo.js)
POST	/auth/signin	Não	Login e obter token (login.js)
GET	/auth/profile	Sim	Mostrar nome do utilizador (tasks.js)
GET	/tasks	Sim	Listar tarefas (tasks.js)
POST	/tasks	Sim	Criar tarefa (tasks.js)
DELETE	/tasks/:id	Sim	Apagar tarefa (tasks.js)

ℹ Nota
Se as rotas do teu backend tiverem paths diferentes (ex: /api/tasks em vez de /tasks), atualiza a constante API e os endpoints nas chamadas fetch em cada ficheiro .js.

5.3 Testar o fluxo completo

Segue esta ordem para testar a aplicação passo a passo:

7. Arranca o backend: npm run dev na pasta do servidor
8. Abre registo.html no browser e cria uma conta nova
9. Confirma que és redirecionado para login.html
10. Faz login com as credenciais criadas
11. Confirma que o token aparece em F12 > Application > Local Storage
12. Na página tasks.html, adiciona algumas tarefas e confirma que aparecem
13. Apaga uma tarefa e confirma que desaparece da lista
14. Clica em Sair e confirma que o token é removido do localStorage
15. Tenta aceder diretamente a tasks.html — deves ser redirecionado para login.html

✓ Dica
Usa o separador Network do F12 para inspecionar cada pedido:
• Em Headers vês o Authorization: Bearer ... a ser enviado
• Em Response vês a resposta completa do servidor
• Um código 200 ou 201 significa sucesso; 401 significa token inválido ou ausente

5.4 Resumo da estrutura final

No final, o projeto frontend deve ter exatamente estes seis ficheiros:

```
frontend-auth/
├── estilo.css      ← Estilos partilhados pelas três páginas
├── registo.html    ← Formulário de criação de conta
├── registo.js      ← Lógica de registo (fetch POST /auth/signup)
├── login.html      ← Formulário de login
├── login.js        ← Lógica de login + guardar token no localStorage
├── tasks.html      ← Interface de gestão de tarefas
└── tasks.js        ← Lógica de tasks (fetch com Authorization header)
```

Critérios de Avaliação

Critério	Peso	Pontuação
`Formulário de registo funcional e ligado à API`	20%	0–20
`Formulário de login com token guardado no localStorage`	25%	0–25
`Listagem de tasks com pedido autenticado`	20%	0–20
`Criação e eliminação de tasks`	15%	0–15
`Proteção da página tasks (redireciona sem token)`	10%	0–10
`Feedback visual de erros e estados de carregamento`	10%	0–10

Entregáveis

- **Pasta frontend-auth** com os seis ficheiros (html, js e css)
- **Repositório Git** com commits ao longo do desenvolvimento
- **Screenshots** do registo, login, lista de tasks e logout

✓ Dica
Bom trabalho!
Agora que percebeste como o token JWT viaja entre o frontend e o backend,
tens a base para qualquer aplicação web autenticada — independentemente da tecnologia.